

KV Store

Esercitazione integrata

Architetture dei Sistemi Distribuiti

Lezione 18

L'esercitazione mette insieme due avanzamenti forti:

- topologia dinamica con `ADD_SHARD` e `REBALANCE`;
- scritture condizionali con `GETV` e `CAS`.

L'obiettivo non e' solo "aggiungere feature". L'obiettivo e' dimostrare di saper progettare un'interfaccia che resti coerente quando:

- il dato si sposta;
- i client osservano versioni;
- piu' operazioni competono sullo stesso stato.

Che Cosa State Ereditando

Dalle lezioni precedenti avete gia' questi ingredienti:

- routing su piu' shard;
- comando WHERE per rendere osservabile il target;
- migrazione esplicita;
- versioni per chiave;
- CAS con fallimento osservabile.

L'esercitazione richiede di combinarli in modo coerente, non di tenerli separati.

Il sistema finale deve almeno supportare:

- GET <key>
- GETV <key>
- SET <key> <value...>
- CAS <key> <expected_version> <value...>
- WHERE <key>
- ADD_SHARD <id> <host> <port>
- REBALANCE

Se aggiungete altro, dovete anche specificarne la semantica.

Prima del codice dovete fissare alcune decisioni di contratto:

- che cosa rappresenta esattamente la versione?
- la versione sopravvive alla migrazione?
- cosa vede il client tra `ADD_SHARD` e `REBALANCE`?
- una CAS letta prima della migrazione resta valida dopo la migrazione?

Se non decidete queste cose prima, il codice verra' contraddittorio.

Il sistema deve rispettare almeno questi vincoli:

- una migrazione non deve perdere ne' valore ne' versione;
- una CAS con versione stantia deve fallire;
- una CAS con versione corretta deve aggiornare sia valore sia versione;
- dopo REBALANCE, routing e posizione reale devono tornare coerenti.

Questi non sono suggerimenti. Sono il nucleo dell'accettazione.

Milestone 1

Primo obiettivo: rendere solido lo store versionato senza migrazione.

- 1 implementare GETV;
- 2 implementare CAS;
- 3 dimostrare un caso di successo;
- 4 dimostrare un `version_mismatch`.

Prima di parlare di shard, dovete poter difendere la semantica locale della versione.

Secondo obiettivo: reintrodurre la topologia dinamica.

- ➊ aggiungere uno shard;
- ➋ rendere visibile il nuovo routing;
- ➌ osservare la divergenza tra target teorico e posizione reale;
- ➍ eseguire il rebalance.

Qui si misura se avete capito che il routing non esaurisce il significato del sistema.

Terzo obiettivo: combinare i due problemi.

- una chiave puo' cambiare shard;
- la sua versione non deve perdere significato;
- una CAS preparata prima dello spostamento deve comportarsi secondo contratto anche dopo lo spostamento.

Questa e' la parte davvero interessante: non state solo migrando dati, state migrando *stato osservabile*.

Avete almeno due direzioni progettuali:

- **router intelligente**: il router conosce topologia, migrazione e inoltro delle operazioni versionate;
- **shard piu' ricchi**: ogni shard conserva coppie (value, version) e il rebalance trasferisce entrambe.

In entrambi i casi dovete spiegare:

- dove vive la versione;
- chi ne garantisce la conservazione;
- chi decide la validita' di una CAS.

Test Di Accettazione Minimi

1. SET k v0
2. GETV k -> versione iniziale
3. CAS k versione v1 -> successo
4. ADD_SHARD ...
5. REBALANCE
6. GETV k -> stessa storia, shard diverso
7. CAS k vecchia v2 -> mismatch

Se questo scenario non e' chiaro o non e' ripetibile, il sistema non e' ancora pronto.

Errori Tipici Che Voglio Vedere Evitati

- migrare il valore ma non la versione;
- resettare la versione quando la chiave cambia shard;
- usare `WHERE` come se implicasse già' la presenza reale del dato;
- trattare `version_mismatch` come errore tecnico invece che come esito del contratto;
- non scrivere test per la finestra transitoria.

Ogni gruppo deve consegnare:

- ① una specifica breve dell'interfaccia;
- ② un'implementazione funzionante;
- ③ test o procedura ripetibile di verifica;
- ④ una nota tecnica sui limiti rimasti aperti.

La nota tecnica conta molto: se il vostro sistema ha limiti dichiarati e coerenti, vale piu' di una soluzione che “sembra funzionare” ma non sa dire che cosa promette.

Valutero' soprattutto:

- chiarezza del contratto;
- correttezza dei casi nominali;
- qualita' dei test sui casi critici;
- capacita' di motivare trade-off e limiti;
- coerenza tra interfaccia dichiarata e comportamento reale.

Questa esercitazione non chiede solo di costruire un sistema piu' grande. Chiede di dimostrare maturita' progettuale:

- separare meccanismo e contratto;
- ragionare su stati transitori;
- rendere difendibili le promesse dell'interfaccia.

Se il codice funziona ma non sapete spiegare cosa promette nei casi difficili, il lavoro non e' completo.